

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: ITA 06

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO4: K- Nearest Neighbour Learning – Locally weighted Regression – Radial Bases Functions – Case Based Learning **(BL 5)**

Assessment Tool Weightage: 20%

QUESTIONS: 03

Total Marks:25

Annexure A

Industry Problem Solving

Code of Conduct:

I VIMALA K 192424276 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

<i>Criteria</i>	<i>Conceptual Understanding</i>	<i>Accuracy of Answers</i>	<i>Application of Knowledge</i>	<i>Completeness of Response</i>	<i>Clarity & Presentation</i>	<i>Total</i>
<i>Weightage</i>	25%	25%	20%	15%	15%	
<i>Q1</i>						
<i>Q2</i>						
<i>Q3</i>						

Signature of the student:

Total Marks :

Vimala K

Annexure A

Short Answer Test

1. A banking system wants to detect potential loan defaulters based on customer financial history and transaction patterns. Design a solution using **K-Nearest Neighbour or Logistic Regression**. Explain how similarity or decision boundaries are used, how features are selected, and how the system adapts to new customer data over time.

Coding:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report

data = {
    "Income": [25000, 50000, 30000, 80000, 45000, 90000, 28000, 60000,
               35000, 75000, 40000, 100000, 55000, 70000, 32000, 85000],

    "CreditScore": [580, 720, 620, 780, 680, 800, 590, 740,
                    640, 760, 670, 820, 650, 750, 600, 790],

    "LoanAmount": [200000, 150000, 250000, 100000, 180000, 120000,
                   220000, 140000, 230000, 110000, 190000, 90000,
                   210000, 130000, 240000, 105000],

    "Transactions": [15, 8, 20, 5, 12, 4, 18, 7,
                     16, 6, 13, 3, 14, 6, 19, 5],

    "Default": [1, 0, 1, 0, 1, 0, 1, 0,
                1, 0, 1, 0, 1, 0, 1, 0]
}

df = pd.DataFrame(data)
X = df[["Income", "CreditScore", "LoanAmount", "Transactions"]]
y = df["Default"]

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
```

```

    test_size=0.25,
    random_state=42,
    stratify=y
)
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)

print("==== LOAN DEFAULT PREDICTION ====")
print("Actual Classes :", y_test.values)
print("Predicted Classes:", y_pred)
print(f"Accuracy      : {accuracy * 100:.2f}%")

print("\nClassification Report:")
print(classification_report(y_test, y_pred, zero_division=0))

new_customer = pd.DataFrame({
    "Income": [55000],
    "CreditScore": [650],
    "LoanAmount": [200000],
    "Transactions": [14]
})

new_customer_scaled = scaler.transform(new_customer)

prediction = model.predict(new_customer_scaled)
probability = model.predict_proba(new_customer_scaled)

print("\n==== NEW CUSTOMER ====")
print(new_customer)

if prediction[0] == 1:
    print("Prediction: Potential Loan Defaulter")
else:
    print("Prediction: Likely Non-Defaulter")

```

```
print(f'Default Probability: {probability[0][1] * 100:.2f}%')
```

```
*** ===== LOAN DEFAULT PREDICTION =====
Actual Classes   : [0 1 0 1]
Predicted Classes: [0 1 0 1]
Accuracy         : 100.00%

Classification Report:
              precision    recall  f1-score   support

     0           1.00        1.00        1.00         2
     1           1.00        1.00        1.00         2

 accuracy          1.00
 macro avg         1.00
 weighted avg      1.00

===== NEW CUSTOMER =====
   Income  CreditScore  LoanAmount  Transactions
0   55000           650    200000             14
Prediction: Potential Loan Defaulter
Default Probability: 81.79%
```

2. A cybersecurity firm wants to identify abnormal network behavior to detect intrusions. Design a system using **KNN or Case-Based Learning**. Explain how similarity between network patterns is measured, how anomalies are detected, and how the model updates with new attack patterns.

Coding:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, classification_report
```

```
data = {
    "PacketRate": [100, 120, 110, 500, 550, 600, 130, 115,
                   520, 125, 580, 105, 140, 570, 135, 590],

    "ConnectionCount": [10, 12, 11, 80, 90, 100, 13, 10,
                       85, 12, 95, 9, 14, 88, 12, 98],
```

```

    "FailedLogins": [1, 0, 1, 20, 25, 30, 0, 1,
                     22, 0, 28, 1, 2, 24, 1, 29],

    "DataTransfer": [200, 220, 210, 900, 950, 1000, 230, 205,
                     920, 215, 980, 190, 240, 940, 225, 990],

    "Intrusion": [0, 0, 0, 1, 1, 1, 0, 0,
                  1, 0, 1, 0, 0, 1, 0, 1]
}

```

```
df = pd.DataFrame(data)
```

```

X = df[
    [
        "PacketRate",
        "ConnectionCount",
        "FailedLogins",
        "DataTransfer"
    ]
]

```

```
y = df["Intrusion"]
```

```

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.25,
    random_state=42,
    stratify=y
)

```

```
scaler = StandardScaler()
```

```

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

```

```
model = KNeighborsClassifier(n_neighbors=3)
```

```
model.fit(X_train, y_train)
```

```
y_pred = model.predict(X_test)
```

```

accuracy = accuracy_score(y_test, y_pred)

print("==== NETWORK INTRUSION DETECTION ====")
print("Actual Classes :", y_test.values)
print("Predicted Classes:", y_pred)
print(f"Accuracy      : {accuracy * 100:.2f}%")

print("\nClassification Report:")
print(classification_report(y_test, y_pred, zero_division=0))

new_activity = pd.DataFrame({
    "PacketRate": [570],
    "ConnectionCount": [88],
    "FailedLogins": [24],
    "DataTransfer": [940]
})

new_activity_scaled = scaler.transform(new_activity)

prediction = model.predict(new_activity_scaled)
probability = model.predict_proba(new_activity_scaled)

print("\n==== NEW NETWORK ACTIVITY ====")
print(new_activity)

if prediction[0] == 1:
    print("Prediction: POSSIBLE INTRUSION DETECTED")
else:
    print("Prediction: NORMAL NETWORK BEHAVIOR")

print(f"Intrusion Probability: {probability[0][1] * 100:.2f}%")

```

```

[2] print( PREDICTION: POSSIBLE INTRUSION DETECTED )
✓ Os else:
    print("Prediction: NORMAL NETWORK BEHAVIOR")

    print(f"Intrusion Probability: {probability[0][1] * 100:.2f}%")

... ===== NETWORK INTRUSION DETECTION =====
Actual Classes : [0 1 0 1]
Predicted Classes: [0 1 0 1]
Accuracy : 100.00%

Classification Report:
              precision    recall  f1-score   support

         0       1.00      1.00      1.00         2
         1       1.00      1.00      1.00         2

   accuracy          1.00      1.00      1.00         4
  macro avg          1.00      1.00      1.00         4
 weighted avg          1.00      1.00      1.00         4

===== NEW NETWORK ACTIVITY =====
PacketRate ConnectionCount FailedLogins DataTransfer
0          570             88           24          940
Prediction: POSSIBLE INTRUSION DETECTED
Intrusion Probability: 100.00%

```

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	25	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	15	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand